

Command	Description
<code>/code-review</code>	Perform a thorough code review. Use when asked to review, check, or audit code. (user)
<code>/impeccable</code>	Use when the user wants to design, redesign, shape, critique, audit, polish, clarify, distill, harden, optimize, adapt, animate, colorize, extract, or otherwise improve a frontend interface. Covers websites, landing pages, dashboards, product UI, app shells, components, forms, settings, onboarding, and empty states. Handles UX review, visual hierarchy, information architecture, cognitive load, accessibility, performance, responsive behavior, theming, anti-patterns, typography, fonts, spacing, layout, alignment, color, motion, micro-interactions, UX c...
<code>/browse</code>	Fast headless browser for QA testing and site dogfooding. Navigate any URL, interact with elements, verify page state, diff before/after actions, take annotated screenshots, check responsive layouts, test forms and uploads, handle dialogs, and assert element states. ~100ms per command. Use when you need to test a feature, verify a deployment, dogfood a user flow, or file a bug with evidence. Use when asked to "open in browser", "test the site", "take a screenshot", or "dogfood this". (gstack) (user)
<code>/update-config</code>	Use this skill to configure the Claude Code harness via settings.json. Automated behaviors ("from now on when X", "each time X", "whenever X", "before/after X") require hooks configured in settings.json - the harness executes these, not Claude, so memory/preferences cannot fulfill them. Also use for: permissions ("allow X", "add permission", "move permission to"), env vars ("set X=Y"), hook troubleshooting, or any changes to settings.json/settings.local.json files. Examples: "allow npm commands", "add bq permission to global settings", "move perm...
<code>/design-consultation</code>	Design consultation: understands your product, researches the landscape, proposes a complete design system (aesthetic, typography, color, layout, spacing, motion), and generates font+color preview pages. Creates DESIGN.md as your project's design source of truth. For existing sites, use /plan-design-review to infer the system instead. Use when asked to "design system", "brand guidelines", or "create DESIGN.md". Proactively suggest when starting a new project's UI with no existing design system or DESIGN.md. (gstack) (user)
<code>/add-dir</code>	Add a new working directory
<code>/advisor</code>	Configure the Advisor Tool to consult a stronger model for guidance at key moments during a task
<code>/agents</code>	Manage agent configurations
<code>/autofix-pr</code>	Monitor and autofix any issues with the current PR (Pull Request)
<code>/background</code>	Continue this session in the background and free the terminal
<code>/branch</code>	Create a branch of the current conversation at this point
<code>/btw</code>	Ask a quick side question without interrupting the main conversation
<code>/chrome</code>	Claude in Chrome (beta) settings
<code>/clear</code>	Start a new session with empty context; previous session stays on disk (resumable with /resume)
<code>/color</code>	Set the prompt bar color for this session
<code>/compact</code>	Free up context by summarizing the conversation so far
<code>/config</code>	Open config panel
<code>/context</code>	Visualize current context usage as a colored grid
<code>/copy</code>	Copy Claude's last response to clipboard (or /copy N for the Nth-latest)
<code>/diff</code>	View uncommitted changes and per-turn diffs
<code>/doctor</code>	Diagnose and verify your Claude Code installation and settings
<code>/effort</code>	Set effort level for model usage
<code>/exit</code>	Exit the CLI
<code>/export</code>	Export the current conversation to a file or clipboard
<code>/extra-usage</code>	Configure extra usage to keep working when limits are hit
<code>/fast</code>	Toggle fast mode (Opus 4.6 only)
<code>/feedback</code>	Submit feedback about Claude Code
<code>/focus</code>	Toggle focus view (show only your prompt, a tool summary, and the final response)
<code>/goal</code>	Set a goal — keep working until the condition is met
<code>/help</code>	Show help and available commands
<code>/hooks</code>	View hook configurations for tool events
<code>/ide</code>	Manage IDE integrations and show status
<code>/install-github-app</code>	Set up Claude GitHub Actions for a repository
<code>/install-slack-app</code>	Install the Claude Slack app
<code>/login</code>	Sign in with your Anthropic account
<code>/logout</code>	Sign out from your Anthropic account
<code>/mcp</code>	Manage MCP servers
<code>/memory</code>	Edit Claude memory files
<code>/mobile</code>	Show QR code to download the Claude mobile app
<code>/model</code>	Set the AI model for Claude Code (currently Sonnet 4.6)
<code>/passes</code>	Share a free week of Claude Code with friends and earn extra usage
<code>/permissions</code>	Manage allow & deny tool permission rules
<code>/plan</code>	Enable plan mode or view the current session plan
<code>/plugin</code>	Manage Claude Code plugins
<code>/powerup</code>	Discover Claude Code features through quick interactive lessons
<code>/privacy-settings</code>	View and update your privacy settings
<code>/radio</code>	Listen to Claude FM lo-fi radio
<code>/recap</code>	Generate a one-line session recap now
<code>/release-notes</code>	View release notes
<code>/reload-plugins</code>	Activate pending plugin changes in the current session
<code>/remote-control</code>	Connect this terminal for remote-control sessions
<code>/remote-env</code>	Configure the default remote environment for teleport sessions
<code>/rename</code>	Rename the current conversation
<code>/resume</code>	Resume a previous conversation
<code>/rewind</code>	Restore the code and/or conversation to a previous point
<code>/skills</code>	List available skills
<code>/status</code>	Show Claude Code status including version, model, account, API connectivity, and tool statuses
<code>/stickers</code>	Order Claude Code stickers
<code>/tasks</code>	List and manage background tasks
<code>/teleport</code>	Resume a Claude Code session from claude.ai
<code>/terminal-setup</code>	Install Shift+Enter key binding for newlines
<code>/theme</code>	Change the theme
<code>/tui</code>	Set the terminal UI renderer (default fullscreen)
<code>/ultraplan</code>	a few minutes · Claude Code on the web drafts a plan you can edit and approve. See https://code.claude.com/docs/en/claude-code-on-the-web
<code>/ultrareview</code>	~5-10 min · Est. cost \$5-\$20 USD · Finds and verifies bugs in your branch. Runs in Claude Code on the web. See https://code.claude.com/docs/en/claude-code-on-the-web
<code>/upgrade</code>	Upgrade to Max for higher rate limits and more Opus
<code>/usage</code>	Show session cost, plan usage, and activity stats
<code>/voice</code>	Toggle voice mode
<code>/web-setup</code>	Setup Claude Code on the web (requires connecting your GitHub account)
<code>/agentic-harness-design</code>	PK (user)
<code>/autoplan</code>	Auto-review pipeline — reads the full CEO, design, eng, and DX review skills from disk and runs them sequentially with auto-decisions using 6 decision principles. Surfaces taste decisions (close approaches, borderline scope, codex disagreements) at a final approval gate. One command, fully reviewed plan out. Use when asked to "auto review", "autoplan", "run all reviews", "review this plan automatically", or "make the decisions for me". Proactively suggest when the user has a plan file and wants to run the full review gauntlet without a
<code>/benchmark</code>	Performance regression detection using the browse daemon. Establishes baselines for page load times, Core Web Vitals, and resource sizes. Compares before/after on every PR. Tracks performance trends over time. Use when: "performance", "benchmark", "page speed", "lighthouse", "web vitals", "bundle size", "load time". (gstack) (user)
<code>/canary</code>	Post-deploy canary monitoring. Watches the live app for console errors, performance regressions, and page failures using the browse daemon. Takes periodic screenshots, compares against pre-deploy baselines, and alerts on anomalies. Use when: "monitor deploy", "canary", "post-deploy check", "watch production", "verify deploy". (gstack) (user)
<code>/careful</code>	Safety guardrails for destructive commands. Warns before rm -rf, DROP TABLE, force-push, git reset --hard, kubectl delete, and similar destructive operations. User can override each warning. Use when touching prod, debugging live systems, or working in a shared environment. Use when asked to "be careful", "safety mode", "prod mode", or "careful mode". (gstack) (user)
<code>/checkpoint</code>	Save and resume working state checkpoints. Captures git state, decisions made, and remaining work so you can pick up exactly where you left off — even across Conductor workspace handoffs between branches. Use when asked to "checkpoint", "save progress", "where was I", "resume", "what was I working on", or "pick up where I left off". Proactively suggest when a session is ending, the user is switching context, or before a long break. (gstack) (user)
<code>/codex</code>	OpenAI Codex CLI wrapper — three modes. Code review: independent diff review via codex review with pass/fail gate. Challenge: adversarial mode that tries to break your code. Consult: ask codex anything with session continuity for follow-ups. The "200 IQ autistic developer" second opinion. Use when asked to "codex review", "codex challenge", "ask codex", "second opinion", or "consult codex". (gstack) (user)

Command	Description
/cso	Chief Security Officer mode. Infrastructure-first security audit: secrets archaeology, dependency supply chain, CI/CD pipeline security, LLM/AI security, skill supply chain scanning, plus OWASP Top 10, STRIDE threat modeling, and active verification. Two modes: daily (zero-noise, 8/10 confidence gate) and comprehensive (monthly deep scan, 2/10 bar). Trend tracking across audit runs. Use when: "security audit", "threat model", "pentest review", "OWASP", "CSO review". (gstack) (user)
/deploy	Run all checks then deploy to production (user)
/design-html	Design finalization: generates production-quality Pretext-native HTML/CSS. Works with approved mockups from /design-shotgun, CEO plans from /plan-ceo-review, design review context from /plan-design-review, or from scratch with a user description. Text actually reflows, heights are computed, layouts are dynamic. 30KB overhead, zero deps. Smart API routing: picks the right Pretext patterns for each design type. Use when: "finalize this design", "turn this into HTML", "build me a page", "implement this design", or after any planning skill. Proactively s...
/design-review	Designer's eye QA: finds visual inconsistency, spacing issues, hierarchy problems, AI slop patterns, and slow interactions — then fixes them. Iteratively fixes issues in source code, committing each fix atomically and re-verifying with before/after screenshots. For plan-mode design review (before implementation), use /plan-design-review. Use when asked to "audit the design", "visual QA", "check if it looks good", or "design polish". Proactively suggest when the user mentions visual inconsistencies or wants to polish the look of a live site. (gstack...
/design-shotgun	Design shotgun: generate multiple AI design variants, open a comparison board, collect structured feedback, and iterate. Standalone design exploration you can run anytime. Use when: "explore designs", "show me options", "design variants", "visual brainstorm", or "I don't like how this looks". Proactively suggest when the user describes a UI feature but hasn't seen what it could look like. (gstack) (user)
/design-taste-frontend	Senior UI/UX Engineer. Architect digital interfaces overriding default LLM biases. Enforces metric-based rules, strict component architecture, CSS hardware acceleration, and balanced design engineering. (user)
/devex-review	Live developer experience audit. Uses the browse tool to actually TEST the developer experience: navigates docs, tries the getting started flow, times TTHW, screenshots error messages, evaluates CLI help text. Produces a DX scorecard with evidence. Compares against
/plan-devex-review	Scores if they exist (the boomerang: plan said 3 minutes, reality says 8). Use when asked to "test the DX", "DX audit", "developer experience test", or "try the onboarding". Proactively suggest after shipping a developer-facing feature. (gstack) (user)
/document-release	Post-ship documentation update. Reads all project docs, cross-references the diff, updates README/ARCHITECTURE/CONTRIBUTING/CLAUDE.md to match what shipped, polishes CHANGELOG voice, cleans up TODOS, and optionally bumps VERSION. Use when asked to "update the docs", "sync documentation", or "post-ship docs". Proactively suggest after a PR is merged or code is shipped. (gstack) (user)
/fix-issue	Investigate and fix a GitHub issue end-to-end (user)
/freeze	Restrict file edits to a specific directory for the session. Blocks Edit and Write outside the allowed path. Use when debugging to prevent accidentally "fixing" unrelated code, or when you want to scope changes to one module. Use when asked to "freeze", "restrict edits", "only edit this folder", or "lock down edits". (gstack) (user)
/full-output-enforcement	Overrides default LLM truncation behavior. Enforces complete code generation, bans placeholder patterns, and handles token-limit splits cleanly. Apply to any task requiring exhaustive, unabridged output. (user)
/gpt-taste	Elite UX/UI & Advanced GSAP Motion Engineer. Enforces Python-driven true randomization for layout variance, strict AIDA page structure, wide editorial typography (bans 6-line wraps), gapless bento grids, strict GSAP ScrollTriggers (pinning, stacking, scrubbing), inline micro-images, and massive section spacing. (user)
/graphify-windows	any input (code, docs, papers, images) → knowledge graph → clustered communities → HTML + JSON + audit report (user)
/gsd-add-tests	Generate tests for a completed phase based on UAT criteria and implementation (user)
/gsd-ai-integration-phase	Generate an AI-SPEC.md design contract for phases that involve building AI systems. (user)
/gsd-audit-fix	Autonomous audit-to-fix pipeline — find issues, classify, fix, test, commit (user)
/gsd-audit-milestone	Audit milestone completion against original intent before archiving (user)
/gsd-audit-uat	Cross-phase audit of all outstanding UAT and verification items (user)
/gsd-autonomous	Run all remaining phases autonomously — discuss→plan→execute per phase (user)
/gsd-capture	Capture ideas, tasks, notes, and seeds to their destination (user)
/gsd-capture	Capture ideas, tasks, notes, and seeds to their destination (user)
/gsd-cleanup	Archive accumulated phase directories from completed milestones (user)
/gsd-code-review	Review source files changed during a phase for bugs, security issues, and code quality problems (user)
/gsd-complete-milestone	Archive completed milestone and prepare for next version (user)
/gsd-config	Configure GSD settings — workflow toggles, advanced knobs, integrations, and model profile (user)
/gsd-debug	Systematic debugging with persistent state across context resets (user)
/gsd-discuss-phase	Gather phase context through adaptive questioning before planning. (user)
/gsd-docs-update	Generate or update project documentation verified against the codebase (user)
/gsd-eval-review	Audit an executed AI phase's evaluation coverage and produce an EVAL-REVIEW.md remediation plan. (user)
/gsd-execute-phase	Execute all plans in a phase with wave-based parallelization (user)
/gsd-explore	Socratic ideation and idea routing — think through ideas before committing to plans (user)
/gsd-extract-learnings	Extract decisions, lessons, patterns, and surprises from completed phase artifacts (user)
/gsd-fast	Execute a trivial task inline — no subagents, no planning overhead (user)
/gsd-forensics	Post-mortem investigation for failed GSD workflows — diagnoses what went wrong. (user)
/gsd-graphify	Build, query, and inspect the project knowledge graph in .planning/graphs/ (user)
/gsd-health	Diagnose planning directory health and optionally repair issues (user)
/gsd-help	Show available GSD commands and usage guide (user)
/gsd-import	Ingest external plans with conflict detection against project decisions before writing anything. (user)
/gsd-inbox	Triage and review open GitHub issues and PRs against project templates and contribution guidelines. (user)
/gsd-ingest-docs	Bootstrap or merge a .planning/ setup from existing ADRs, PRDs, SPECS, and docs in a repo. (user)
/gsd-manager	Interactive command center for managing multiple phases from one terminal (user)
/gsd-map-codebase	Analyze codebase with parallel mapper agents to produce .planning/codebase/ documents (user)
/gsd-milestone-summary	Generate a comprehensive project summary from milestone artifacts for team onboarding and review (user)
/gsd-mvp-phase	Plan a phase as a vertical MVP slice — user story, SPIDR splitting, then plan-phase (user)
/gsd-new-milestone	Start a new milestone cycle — update PROJECT.md and route to requirements (user)
/gsd-new-project	Initialize a new project with deep context gathering and PROJECT.md (user)
/gsd-ns-context	codebase intelligence map graphify docs learnings (user)
/gsd-ns-ideate	exploration capture explore sketch spike spec capture (user)
/gsd-ns-manage	config workspace workstreams thread update ship inbox (user)
/gsd-ns-project	project lifecycle milestones audits summary (user)
/gsd-ns-review	quality gates code review debug audit security eval ui (user)
/gsd-ns-workflow	workflow discuss plan execute verify phase progress (user)
/gsd-pause-work	Create context handoff when pausing work mid-phase (user)
/gsd-phase	CRUD for phases in ROADMAP.md — add, insert, remove, or edit phases (user)
/gsd-plan-phase	Create detailed phase plan (PLAN.md) with verification loop (user)
/gsd-plan-review-convergence	Cross-AI plan convergence loop — replan with review feedback until no HIGH concerns remain. (user)
/gsd-pr-branch	Create a clean PR branch by filtering out .planning/ commits — ready for code review (user)
/gsd-profile-user	Generate developer behavioral profile and create Claude-discoverable artifacts (user)
/gsd-progress	Check progress, advance workflow, or dispatch freeform intent — the unified GSD situational command (user)
/gsd-quick	Execute a quick task with GSD guarantees (atomic commits, state tracking) but skip optional agents (user)
/gsd-resume-work	Resume work from previous session with full context restoration (user)
/gsd-review	Request cross-AI peer review of phase plans from external AI CLIs (user)
/gsd-review-backlog	Review and promote backlog items to active milestone (user)
/gsd-secure-phase	Retroactively verify threat mitigations for a completed phase (user)
/gsd-settings	Configure GSD workflow toggles and model profile (user)
/gsd-ship	Create PR, run review, and prepare for merge after verification passes (user)
/gsd-sketch	Sketch UI/design ideas with throwaway HTML mockups, or propose what to sketch next (frontier mode) (user)
/gsd-spec-phase	Clarify WHAT a phase delivers with ambiguity scoring; produces a SPEC.md before discuss-phase. (user)
/gsd-spike	Spike an idea through experiential exploration, or propose what to spike next (frontier mode) (user)
/gsd-stats	Display project statistics — phases, plans, requirements, git metrics, and timeline (user)
/gsd-thread	Manage persistent context threads for cross-session work (user)
/gsd-ui-phase	Generate UI design contract (UI-SPEC.md) for frontend phases (user)
/gsd-ui-review	Retroactive 6-pillar visual audit of implemented frontend code (user)
/gsd-ultraplan-phase	[BETA] Offload plan phase to Claude Code's ultraplan cloud; review in browser and import back. (user)
/gsd-undo	Safe git revert. Roll back phase or plan commits using the phase manifest with dependency checks. (user)
/gsd-update	Update GSD to latest version with changelog display (user)
/gsd-validate-phase	Retroactively audit and fill Nyquist validation gaps for a completed phase (user)
/gsd-verify-work	Validate built features through conversational UAT (user)

Command	Description
<code>/gsd-workspace</code>	Manage GSD workspaces — create, list, or remove isolated workspace environments (user)
<code>/gsd-workstreams</code>	Manage parallel workstreams — list, create, switch, status, progress, complete, and resume (user)
<code>/gstack</code>	Fast headless browser for QA testing and site dogfooding. Navigate pages, interact with elements, verify state, diff before/after, take annotated screenshots, test responsive layouts, forms, uploads, dialogs, and capture bug evidence. Use when asked to open or test a site, verify a deployment, dogfood a user flow, or file a bug with screenshots. (gstack) (user)
<code>/gstack-upgrade</code>	Upgrade gstack to the latest version. Detects global vs vendored install, runs the upgrade, and shows what's new. Use when asked to "upgrade gstack", "update gstack", or "get latest version". (user)
<code>/guard</code>	Full safety mode: destructive command warnings + directory-scoped edits. Combines /careful (warns before rm -rf, DROP TABLE, force-push, etc.) with /freeze (blocks edits outside a specified directory). Use for maximum safety when touching prod or debugging live systems. Use when asked to "guard mode", "full safety", "lock it down", or "maximum safety". (gstack) (user)
<code>/health</code>	Code quality dashboard. Wraps existing project tools (type checker, linter, test runner, dead code detector, shell linter), computes a weighted composite 0-10 score, and tracks trends over time. Use when: "health check", "code quality", "how healthy is the codebase", "run all checks", "quality score". (gstack) (user)
<code>/high-end-visual-design</code>	Teaches the AI to design like a high-end agency. Defines the exact fonts, spacing, shadows, card structures, and animations that make a website feel expensive. Blocks all the common defaults that make AI designs look cheap or generic. (user)
<code>/humanizer</code>	Remove signs of AI-generated writing from text. Use when editing or reviewing text to make it sound more natural and human-written. Based on Wikipedia's comprehensive "Signs of AI writing" guide. Detects and fixes patterns including: inflated symbolism, promotional language, superficial -ing analyses, vague attributions, em dash overuse, rule of three, AI vocabulary words, negative parallelisms, and excessive conjunctive phrases. (user)
<code>/industrial-brutalist-ui</code>	Raw mechanical interfaces fusing Swiss typographic print with military terminal aesthetics. Rigid grids, extreme type scale contrast, utilitarian color, analog degradation effects. For data-heavy dashboards, portfolios, or editorial sites that need to feel like declassified blueprints. (user)
<code>/investigate</code>	Systematic debugging with root cause investigation. Four phases: investigate, analyze, hypothesize, implement. Iron Law: no fixes without root cause. Use when asked to "debug this", "fix this bug", "why is this broken", "investigate this error", or "root cause analysis". Proactively invoke this skill (do NOT debug directly) when the user reports errors, 500 errors, stack traces, unexpected behavior, "it was working yesterday", or is troubleshooting why something stopped working. (gstack) (user)
<code>/land-and-deploy</code>	Land and deploy workflow. Merges the PR, waits for CI and deploy, verifies production health via canary checks. Takes over after /ship creates the PR. Use when: "merge", "land", "deploy", "merge and verify", "land it", "ship it to production". (gstack) (user)
<code>/learn</code>	Manage project learnings. Review, search, prune, and export what gstack has learned across sessions. Use when asked to "what have we learned", "show learnings", "prune stale learnings", or "export learnings". Proactively suggest when the user asks about past patterns or wonders "didn't we fix this before?" (user)
<code>/minimalist-ui</code>	Clean editorial-style interfaces. Warm monochrome palette, typographic contrast, flat bento grids, muted pastels. No gradients, no heavy shadows. (user)
<code>/office-hours</code>	YC Office Hours — two modes. Startup mode: six forcing questions that expose demand reality, status quo, desperate specificity, narrowest wedge, observation, and future-fit. Builder mode: design thinking brainstorming for side projects, hackathons, learning, and open source. Saves a design doc. Use when asked to "brainstorm this", "I have an idea", "help me think through this", "office hours", or "is this worth building". Proactively invoke this skill (do NOT answer directly) when the user describes a new product idea, asks whether something is wor...
<code>/open-gstack-browser</code>	Launch GStack Browser — AI-controlled Chromium with the sidebar extension baked in. Opens a visible browser window where you can watch every action in real time. The sidebar shows a live activity feed and chat. Anti-bot stealth built in. Use when asked to "open gstack browser", "launch browser", "connect chrome", "open chrome", "real browser", "launch chrome", "side panel", or "control my browser". (user)
<code>/open-gstack-browser</code>	Launch GStack Browser — AI-controlled Chromium with the sidebar extension baked in. Opens a visible browser window where you can watch every action in real time. The sidebar shows a live activity feed and chat. Anti-bot stealth built in. Use when asked to "open gstack browser", "launch browser", "connect chrome", "open chrome", "real browser", "launch chrome", "side panel", or "control my browser". (user)
<code>/pair-agent</code>	Pair a remote AI agent with your browser. One command generates a setup key and prints instructions the other agent can follow to connect. Works with OpenClaw, Hermes, Codex, Cursor, or any agent that can make HTTP requests. The remote agent gets its own tab with scoped access (read+write by default, admin on request). Use when asked to "pair agent", "connect agent", "share browser", "remote browser", "let another agent use my browser", or "give browser access". (gstack) (user)
<code>/paul:add-phase</code>	Add a new phase to current milestone (user)
<code>/paul:apply</code>	Execute an approved PLAN (user)
<code>/paul:assumptions</code>	Surface Claude's assumptions about a phase before planning (user)
<code>/paul:audit</code>	Run enterprise-grade architectural audit on current plan (user)
<code>/paul:complete-milestone</code>	Mark current milestone as complete (user)
<code>/paul:config</code>	<objective> (user)
<code>/paul:consider-issues</code>	Review deferred issues with codebase context, triage and route (user)
<code>/paul:discover</code>	Research technical options before planning a phase (user)
<code>/paul:discuss</code>	Explore and articulate phase vision before planning (user)
<code>/paul:discuss-milestone</code>	Explore and articulate next milestone vision (user)
<code>/paul:flows</code>	Configure specialized workflow integrations (user)
<code>/paul:handoff</code>	Generate comprehensive session handoff document (user)
<code>/paul:help</code>	Show available PAUL commands and usage guide (user)
<code>/paul:init</code>	Initialize PAUL in a project with conversational setup (user)
<code>/paul:map-codebase</code>	<objective> (user)
<code>/paul:milestone</code>	Create a new milestone in the project (user)
<code>/paul:pause</code>	Create handoff file and prepare for session break (user)
<code>/paul:plan</code>	Enter PLAN phase for current or new plan (user)
<code>/paul:plan-fix</code>	Plan fixes for UAT issues from verify (user)
<code>/paul:progress</code>	Smart status with routing - suggests ONE next action (user)
<code>/paul:register</code>	Generate paul.json for an existing PAUL project that predates v1.1 satellite manifest support (user)
<code>/paul:remove-phase</code>	Remove a future (not started) phase (user)
<code>/paul:research</code>	Research a topic using subagents for discovery (user)
<code>/paul:research-phase</code>	Research unknowns for a phase using subagents (user)
<code>/paul:resume</code>	Restore context from handoff and continue work (user)
<code>/paul:status</code>	[DEPRECATED] Use /paul:progress instead (user)
<code>/paul:unify</code>	Reconcile plan vs actual and close the loop (user)
<code>/paul:verify</code>	Guide manual user acceptance testing of recently built features (user)
<code>/plan-ceo-review</code>	CEO/founder-mode plan review. Rethink the problem, find the 10-star product, challenge premises, expand scope when it creates a better product. Four modes: SCOPE EXPANSION (dream big), SELECTIVE EXPANSION (hold scope + cherry-pick expansions), HOLD SCOPE (maximum rigor), SCOPE REDUCTION (strip to essentials). Use when asked to "think bigger", "expand scope", "strategy review", "rethink this", or "is this ambitious enough". Proactively suggest when the user is questioning scope or ambition of a plan, or when the plan feels like it could be thinking bi...
<code>/plan-design-review</code>	Designer's eye plan review — interactive, like CEO and Eng review. Rates each design dimension 0-10, explains what would make it a 10, then fixes the plan to get there. Works in plan mode. For live site visual audits, use /design-review. Use when asked to "review the design plan" or "design critique". Proactively suggest when the user has a plan with UI/UX components that should be reviewed before implementation. (gstack) (user)
<code>/plan-devex-review</code>	Interactive developer experience plan review. Explores developer personas, benchmarks against competitors, designs magical moments, and traces friction points before scoring. Three modes: DX EXPANSION (competitive advantage), DX POLISH (bulletproof every touchpoint), DX TRIAGE (critical gaps only). Use when asked to "DX review", "developer experience audit", "devex review", or "API design review". Proactively suggest when the user has a plan for developer-facing products (APIs, CLIs, SDKs, libraries, platforms, docs). (gstack) (user)
<code>/plan-eng-review</code>	Eng manager-mode plan review. Lock in the execution plan — architecture, data flow, diagrams, edge cases, test coverage, performance. Walks through issues interactively with opinionated recommendations. Use when asked to "review the architecture", "engineering review", or "lock in the plan". Proactively suggest when the user has a plan or design doc and is about to start coding — to catch architecture issues before implementation. (gstack) (user)
<code>/pr-review</code>	Full code review on current branch vs main before raising a PR (user)
<code>/product-management:brainstorm</code>	Brainstorm a product idea, problem space, or strategic question with a sharp thinking partner (user)
<code>/qa</code>	Systematically QA test a web application and fix bugs found. Runs QA testing, then iteratively fixes bugs in source code, committing each fix atomically and re-verifying. Use when asked to "qa", "QA", "test this site", "find bugs", "test and fix", or "fix what's broken". Proactively suggest when the user says a feature is ready for testing or asks "does this work?". Three tiers: Quick (critical/high only), Standard (+ medium), Exhaustive (+ cosmetic). Produces before/after health scores, fix evidence, and a ship-readiness summary. For report-on...
<code>/qa-only</code>	Report-only QA testing. Systematically tests a web application and produces a structured report with health score, screenshots, and repro steps — but never fixes anything. Use when asked to "just report bugs", "qa report only", or "test but don't fix". For the full test-fix-verify loop, use /qa instead. Proactively suggest when the user wants a bug report without any code changes. (gstack) (user)
<code>/redesign-existing-projects</code>	Upgrades existing websites and apps to premium quality. Audits current design, identifies generic AI patterns, and applies high-end design standards without breaking functionality. Works with any CSS framework or vanilla CSS. (user)
<code>/refactor</code>	Safely refactor code to improve quality without changing behaviour. Use when asked to refactor, clean up, or improve code. (user)
<code>/retro</code>	Weekly engineering retrospective. Analyzes commit history, work patterns, and code quality metrics with persistent history and trend tracking. Team-aware: breaks down per-person contributions with praise and growth areas. Use when asked to "weekly retro", "what did we ship", or "engineering retrospective". Proactively suggest at the end of a work week or sprint. (gstack) (user)

Command	Description
<code>/review</code>	Pre-landing PR review. Analyzes diff against the base branch for SQL safety, LLM trust boundary violations, conditional side effects, and other structural issues. Use when asked to "review this PR", "code review", "pre-landing review", or "check my diff". Proactively suggest when the user is about to merge or land code changes. (gstack) (user)
<code>/security-audit</code>	Run a security audit of specified files or the whole codebase. Use for security reviews, vulnerability checks. (user)
<code>/setup-browser-cookies</code>	Import cookies from your real Chromium browser into the headless browse session. Opens an interactive picker UI where you select which cookie domains to import. Use before QA testing authenticated pages. Use when asked to "import cookies", "login to the site", or "authenticate the browser". (gstack) (user)
<code>/setup-deploy</code>	Configure deployment settings for <code>/land-and-deploy</code> . Detects your deploy platform (Fly.io, Render, Vercel, Netlify, Heroku, GitHub Actions, custom), production URL, health check endpoints, and deploy status commands. Writes the configuration to <code>CLAUDE.md</code> so all future deploys are automatic. Use when: "setup deploy", "configure deployment", "set up land-and-deploy", "how do I deploy with gstack", "add deploy config". (user)
<code>/ship</code>	Ship workflow: detect + merge base branch, run tests, review diff, bump VERSION, update CHANGELOG, commit, push, create PR. Use when asked to "ship", "deploy", "push to main", "create a PR", "merge and push", or "get it deployed". Proactively invoke this skill (do NOT push/PR directly) when the user says code is ready, asks about deploying, wants to push code up, or asks to create a PR. (gstack) (user)
<code>/stitch-design-taste</code>	Semantic Design System Skill for Google Stitch. Generates agent-friendly DESIGN.md files that enforce premium, anti-generic UI standards — strict typography, calibrated color, asymmetric layouts, perpetual micro-motion, and hardware-accelerated performance. (user)
<code>/subagent-catalog:fetch</code>	Fetch full subagent definition from catalog. Use when user wants to get, download, view, or use a specific subagent. (user)
<code>/subagent-catalog:invalidate</code>	Invalidate the subagent-catalog cache. Use when results seem stale or user explicitly asks to refresh or clear cache. (user)
<code>/subagent-catalog:list</code>	List all categories and agents in the subagent catalog. Use when user wants to see everything available or browse the full catalog. (user)
<code>/subagent-catalog:README</code>	subagent-catalog (user)
<code>/subagent-catalog:search</code>	Search the awesome-claude-code-subagents catalog. Use when user wants to find, discover, or browse available subagents by name, category, or capability. (user)
<code>/supabase</code>	Use when doing ANY task involving Supabase. Triggers: Supabase products (Database, Auth, Edge Functions, Realtime, Storage, Vectors, Cron, Queues); client libraries and SSR integrations (supabase-js, @supabase/ssr) in Next.js, React, SvelteKit, Astro, Remix; auth issues (login, logout, sessions, JWT, cookies, getSession, getUser, getClaims, RLS); Supabase CLI or MCP server; schema changes, migrations, security audits, Postgres extensions (pg_graphql, pg_cron, pg_vector). (user)
<code>/supabase-postgres-best-practices</code>	Postgres performance optimization and best practices from Supabase. Use this skill when writing, reviewing, or optimizing Postgres queries, schema designs, or database configurations. (user)
<code>/uncodifyx</code>	Prevents generic AI/Codex UI patterns when generating frontend code. Use this skill whenever generating HTML, CSS, React, Vue, Svelte, or any frontend UI code to enforce clean, human-designed aesthetics inspired by Linear, Raycast, Stripe, and GitHub instead of typical AI-generated UI. (user)
<code>/unfreeze</code>	Clear the freeze boundary set by <code>/freeze</code> , allowing edits to all directories again. Use when you want to widen edit scope without ending the session. Use when asked to "unfreeze", "unlock edits", "remove freeze", or "allow all edits". (gstack) (user)
<code>/ai-gateway</code>	(vercel) Vercel AI Gateway expert guidance. Use when configuring model routing, provider failover, cost tracking, or managing multiple AI providers through a unified API.
<code>/ai-sdk</code>	(vercel) Vercel AI SDK expert guidance. Use when building AI-powered features — chat interfaces, text generation, structured output, tool calling, agents, MCP integration, streaming, embeddings, reranking, image generation, or working with any LLM provider.
<code>/auth</code>	(vercel) Authentication integration guidance — Clerk (native Vercel Marketplace), Descope, and Auth0 setup for Next.js applications. Covers middleware auth patterns, sign-in/sign-up flows, and Marketplace provisioning. Use when implementing user authentication.
<code>/batch</code>	Research and plan a large-scale change, then execute it in parallel across 5–30 isolated worktree agents that each open a PR.
<code>/chat-sdk</code>	(vercel) Vercel Chat SDK expert guidance. Use when building multi-platform chat bots — Slack, Telegram, Microsoft Teams, Discord, Google Chat, GitHub, Linear — with a single codebase. Covers the Chat class, adapters, threads, messages, cards, modals, streaming, state management, and webhook setup.
<code>/claude-api</code>	Build, debug, and optimize Claude API / Anthropic SDK apps. Apps built with this skill should include prompt caching. Also handles migrating existing Claude API code between Claude model versions (4.5 → 4.6, 4.6 → 4.7, retired-model replacements). TRIGGER when: code imports `anthropic` / `@anthropic-ai/sdk` ; user asks for the Claude API, Anthropic SDK, or Managed Agents; user adds/modifies/tunes a Claude feature (caching, thinking, compaction, tool use, batch, files, citations, memory) or model (Opus/Sonnet/Haiku) in a file; questions about prompt c...
<code>/debug</code>	Enable debug logging for this session and help diagnose issues
<code>/deployments-cicd</code>	(vercel) Vercel deployment and CI/CD expert guidance. Use when deploying, promoting, rolling back, inspecting deployments, building with --prebuilt, or configuring CI workflow files for Vercel.
<code>/env-vars</code>	(vercel) Vercel environment variable expert guidance. Use when working with .env files, vercel env commands, OIDC tokens, or managing environment-specific configuration.
<code>/fewer-permission-prompts</code>	Scan your transcripts for common read-only Bash and MCP tool calls, then add a prioritized allowlist to project.claude/settings.json to reduce permission prompts.
<code>/init</code>	Initialize a new CLAUDE.md file with codebase documentation
<code>/insights</code>	Generate a report analyzing your Claude Code sessions
<code>/knowledge-update</code>	(vercel) Corrects outdated LLM knowledge about the Vercel platform and introduces new products. Injected at session start.
<code>/loop</code>	Run a prompt or slash command on a recurring interval (e.g. /loop 5m /foo). Omit the interval to let the model self-pace.
<code>/next-cache-components</code>	(vercel) Next.js 16 Cache Components guidance — PPR, use cache directive, cacheLife, cacheTag, updateTag, and migration from unstable_cache. Use when implementing partial prerendering, caching strategies, or migrating from older Next.js cache patterns.
<code>/next-forge</code>	(vercel) next-forge expert guidance — production-grade Turborepo monorepo SaaS starter by Vercel. Use when working in a next-forge project, scaffolding with `npx next-forge init`, or editing @repo/* workspace packages.
<code>/next-upgrade</code>	(vercel) Upgrade Next.js to the latest version following official migration guides and codemods. Use when upgrading Next.js versions, running codemods, or migrating between major releases.
<code>/nextjs</code>	(vercel) Next.js App Router expert guidance. Use when building, debugging, or architecting Next.js applications — routing, Server Components, Server Actions, Cache Components, layouts, middleware/proxy, data fetching, rendering strategies, and deployment on Vercel.
<code>/react-best-practices</code>	(vercel) React best-practices reviewer for TSX files. Triggers after editing multiple TSX components to run a condensed quality checklist covering component structure, hooks usage, accessibility, performance, and TypeScript patterns.
<code>/routing-middleware</code>	(vercel) Vercel Routing Middleware guidance — request interception before cache, rewrites, redirects, personalization. Works with any framework. Supports Edge, Node.js, and Bun runtimes. Use when intercepting requests at the platform level.
<code>/runtime-cache</code>	(vercel) Vercel Runtime Cache API guidance — ephemeral per-region key-value cache with tag-based invalidation. Shared across Functions, Routing Middleware, and Builds. Use when implementing caching strategies beyond framework-level caching.
<code>/schedule</code>	Create, update, list, or run scheduled remote agents (routines) that execute on a cron schedule.
<code>/security-review</code>	Complete a security review of the pending changes on the current branch
<code>/shadcn</code>	(vercel) shadcn/ui expert guidance — CLI, component installation, composition patterns, custom registries, theming, Tailwind CSS integration, and high-quality interface design. Use when initializing shadcn, adding components, composing product UI, building custom registries, configuring themes, or troubleshooting component issues.
<code>/simplify</code>	Review changed code for reuse, quality, and efficiency, then fix any issues found.
<code>/statusline</code>	Set up Claude Code's status line UI
<code>/team-onboarding</code>	Help teammates ramp on Claude Code with a guide from your usage
<code>/turbopack</code>	(vercel) Turbopack expert guidance. Use when configuring the Next.js bundler, optimizing HMR, debugging build issues, or understanding the Turbopack vs Vitepack differences.
<code>/vercel-agent</code>	(vercel) Vercel Agent guidance — AI-powered code review, incident investigation, and SDK installation. Automates PR analysis and anomaly debugging. Use when configuring or understanding Vercel's AI development tools.
<code>/vercel-cli</code>	(vercel) Vercel CLI expert guidance. Use when deploying, managing environment variables, linking projects, viewing logs, managing domains, or interacting with the Vercel platform from the command line.
<code>/vercel-functions</code>	(vercel) Vercel Functions expert guidance — Serverless Functions, Edge Functions, Fluid Compute, streaming, Cron Jobs, and runtime configuration. Use when configuring, debugging, or optimizing server-side code running on Vercel.
<code>/vercel-sandbox</code>	(vercel) Vercel Sandbox guidance — ephemeral Firecracker microVMs for running untrusted code safely. Supports AI agents, code generation, and experimentation. Use when executing user-generated or AI-generated code in isolation.
<code>/vercel-storage</code>	(vercel) Vercel storage expert guidance — Blob, Edge Config, and Marketplace storage (Neon Postgres, Upstash Redis). Use when choosing, configuring, or using data storage with Vercel applications.
<code>/vercel:bootstrap</code>	(vercel) Bootstrap a repository with Vercel-linked resources by running preflight checks, provisioning integrations, verifying env keys, and then executing db/dev startup commands safely.
<code>/vercel:deploy</code>	(vercel) Deploy the current project to Vercel. Pass "prod" or "production" as argument to deploy to production. Default is preview deployment.
<code>/vercel:env</code>	(vercel) Manage Vercel environment variables. Commands include list, pull, add, remove, and diff. Use to sync environment variables between Vercel and your local development environment.
<code>/vercel:marketplace</code>	(vercel) Discover and install Vercel Marketplace integrations. Use to find databases, CMS, auth providers, and other services available on the Vercel Marketplace.
<code>/vercel:status</code>	(vercel) Show the status of the current Vercel project — recent deployments, linked project info, and environment overview.
<code>/verification</code>	(vercel) Full-story verification — infers what the user is building, then verifies the complete flow end-to-end: browser → API → data → response. Triggers on dev server start and 'why isn't this working' signals.
<code>/workflow</code>	(vercel) Vercel Workflow DevKit (WDK) expert guidance. Use when building durable workflows, long-running tasks, API routes or agents that need pause/resume, retries, step-based execution, or crash-safe orchestration with Vercel Workflow.